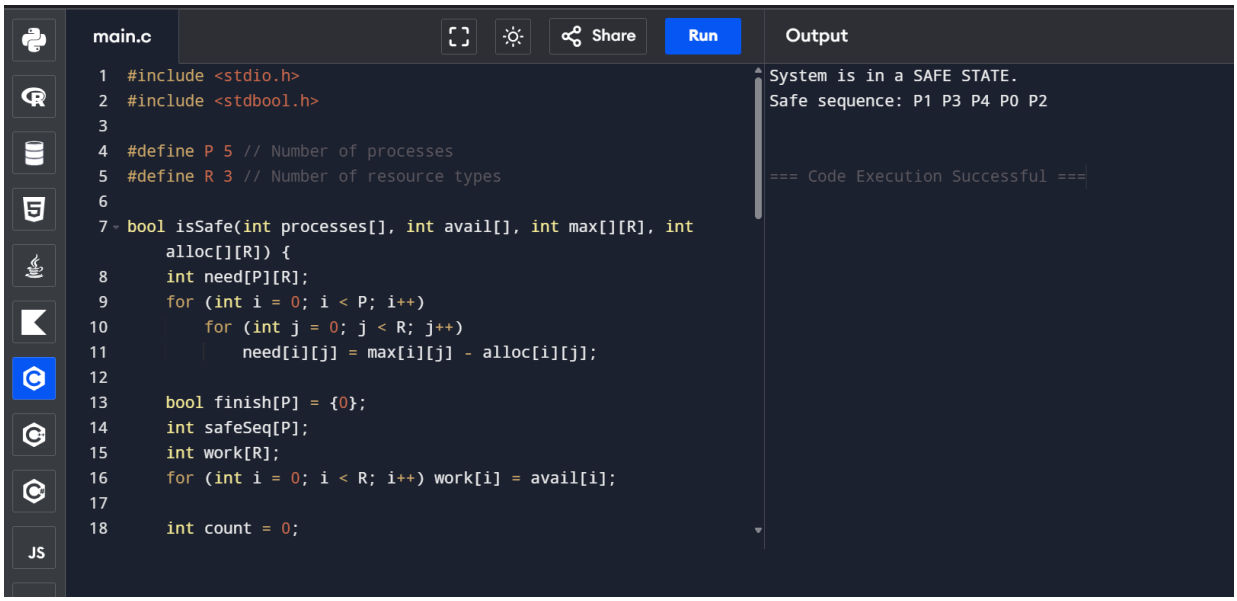
EXPERIMENT 17:

17. Illustrate the deadlock avoidance concept by simulating Banker's algorithm with C program .



PSEUDO CODE :

Algorithm Banker (Allocation, Max, Available, Need, n, m):
1. Compute Need[i][j] = Max[i][j] - Allocation[i][j] for all i, j.
2. Initialize Work = Available and Finish[i] = false for i = 0..n-1.
3. Find an index i such that:
   a. Finish[i] == false
   b. Need[i][j] <= Work[j] for all resources j = 0..m-1
4. If such an i exists:
   - Work[j] = Work[j] + Allocation[i][j]
   - Finish[i] = true
   - Append i to safeSequence
   - Repeat Step 3.
5. If Finish[i] == true for all i:
   - System is in a SAFE STATE.
   - Return safeSequence.
6. Else:
   - System is in an UNSAFE STATE (Deadlock risk).

CODE:

```
#include <stdio.h>
#include <stdbool.h>

#define P 5 // Number of processes
#define R 3 // Number of resource types

bool isSafe(int processes[], int avail[], int max[][R], int alloc[][R]) {
    int need[P][R];
    for (int i = 0; i < P; i++)
        for (int j = 0; j < R; j++)
            need[i][j] = max[i][j] - alloc[i][j];

    bool finish[P] = {0};
    int safeSeq[P];
    int work[R];
    for (int i = 0; i < R; i++) work[i] = avail[i];

    int count = 0;
    while (count < P) {
        bool found = false;
        for (int p = 0; p < P; p++) {
            if (!finish[p]) {
                int j;
```

```
            for (j = 0; j < R; j++)
                if (need[p][j] > work[j]) break;

            if (j == R) {
                for (int k = 0; k < R; k++) work[k] += alloc[p][k];
                safeSeq[count++] = p;
                finish[p] = true;
                found = true;
            }
        }
    }
    if (!found) {
        printf("System is NOT in a safe state.\n");
        return false;
    }
  }

  printf("System is in a SAFE STATE.\nSafe sequence: ");
  for (int i = 0; i < P; i++) printf("P%d ", safeSeq[i]);
  printf("\n");
  return true;
}

int main() {
  int processes[] = {0, 1, 2, 3, 4};
```

```
    int avail[] = {3, 3, 2};
    int max[P][R] = {{7, 5, 3}, {3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3}};
    int alloc[P][R] = {{0, 1, 0}, {2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0, 2}};

    isSafe(processes, avail, max, alloc);
    return 0;
}
```